

How can a model finish a multi-step task?

A recursive language model (RLM) can use tools and ask helper models to solve parts of a task.

For this study, we returned to a basic skill.

Can a small model look up missing information and use it to finish a task?

Example: Make a wooden pickaxe in a crafting game.

1. The model asks which ingredients it needs.
2. It makes the parts and then the pickaxe.
3. The game checks whether the pickaxe actually exists.

Simplified illustration of TextCraft, our test environment. These experiments use one model with tools, not recursive helper calls.

We compared two kinds of worked examples.

A program generated two kinds of worked examples. We trained copies of the same model to imitate their actions. This is **supervised fine-tuning**.

The teacher already knows the recipes.

It chooses the route using that knowledge, then shows the steps.

“Look up planks. Make planks.
Continue along the planned route.”

The teacher discovers the recipes.

It chooses the route by looking up which parts are needed.

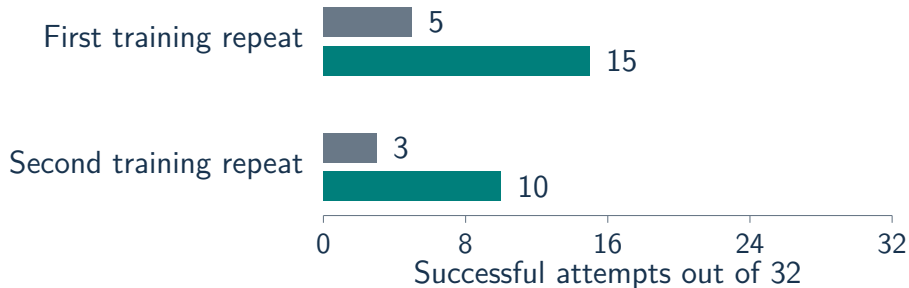
“Look up the pickaxe. Discover its parts. Look up their recipes.”

Both show recipe lookups. They differ in how the route is chosen.

Simplified examples, not literal training records. Both use the same 32 training tasks and 366 demonstrated steps. Their histories and amount of text differ.

Examples that discover the route worked better.

We trained each kind of model twice and tested them on the same tasks.



■ Teacher knows recipes ■ Teacher discovers recipes

Sixteen evaluation tasks, two attempts each. The original examples were corrected for a known quantity error. This correction did not remove the advantage.

On changed recipes, discovery still led.

We changed the recipes and starting supplies, then tested both training repeats. Each number is successes out of 16 attempts.

Training examples	First repeat	Second repeat
Teacher knows the recipes	2	1
Same teacher, with the quantity mistake fixed	4	1
Teacher discovers the recipes	6	8

The lead was small in one repeat and large in the other.

All outcomes are known. The first repeat is too uncertain to establish a clear advantage over corrected examples; the second gives stronger evidence. These are still small tests within one kind of game.

Letting code fill in recipe details helped.

The model still chooses the item and amount. Code fills in ingredients from a recipe the model already looked up.

We used the discovery-trained models, with **no extra training**.

Test	Model writes the ingredients	Code fills the ingredients
Slide 4 setting, first trained model	6/16	12/16
Slide 4 setting, second trained model	8/16	10/16
Another recipe setting, first model	9/16	14/16

Question: Which decisions need a model, and which details should code handle?

Numbers are successful attempts. All outcomes are known. This is a small, preliminary test within one game; it does not demonstrate better recursive planning.